

Проект Яндекс Лицей

Общая идея

Миниатюрный аналог заметок. Заметки можно не только хранить, но и **забирать оттуда** в том виде, который удобен: как простой текст, как красивый HTML-файл или как Markdown-документ. Плюс — можно прикрепить картинку и **получить к своим заметкам доступ через REST API**, если захочешь написать своего бота или мобильное приложение.

Что видит и делает пользователь

1. Вход в систему

Пользователь заходит на сайт. Его встречает страница со ссылками «Войти» и «Зарегистрироваться». После регистрации (логин + пароль) он попадает в свой личный кабинет с заметками (чужие заметки не видны).

2. Список заметок

Главный экран после входа — это список всех созданных заметок. Каждая заметка отображается в виде карточки или строки в таблице. Видно заголовок, дату последнего изменения. Тут же — кнопки:

-  Редактировать
-  Удалить
-  Экспорт (выпадающий список: MD / HTML / TXT)

Плюс большая кнопка « Новая заметка».

3. Создание и редактирование заметки

Открывается форма с двумя полями:

- **Заголовок** (обычное текстовое поле)
- **Текст заметки** (большое поле, где можно писать Markdown — `# заголовок` , `**жирный**` , списки, ссылки)

Тут же — блок «Прикрепить файлы»:

- Кнопка «Выбрать изображение» (PNG / JPEG / GIF, до 5 МБ)
- После загрузки файлы показываются миниатюрами под формой

Кнопка «Сохранить» — и заметка готова.

4. Просмотр заметки

Отдельная страница, где:

- Заголовок крупно
- Текст заметки, преобразованный из Markdown в нормальный HTML (заголовки, списки, форматирование)
- Внизу — прикреплённые изображения (можно кликнуть, чтобы открыть в полный размер)
- Кнопки для скачивания в 3 форматах

5. Экспорт — это самое интересное

Пользователь нажимает «Экспорт» и выбирает формат:

- **MD (Markdown)** — скачивается `.md` файл с исходной разметкой, которую писал пользователь. Идеально для блогов, GitHub, Obsidian.
- **HTML** — браузер скачивает `.html` файл, который можно открыть на любом компьютере. Текст будет выглядеть красиво (встроенный простой CSS) — заголовки, отступы, картинки (они идут отдельными файлами или встраиваются base64, решаем как проще).
- **TXT** — из Markdown убирается вся разметка (удаляются `#`, `**`, `-` и т.д.), остаётся чистый читаемый текст. Идеально для заметок в блокнот.

При экспорте вложения (картинки) либо игнорируются (в TXT), либо прилагаются как есть (в HTML — картинки ссылаются на оригиналы на сервере).

REST API (не для всех, но для тех, кому надо)

Если пользователь — разработчик или ему нужно автоматизировать работу с заметками, он может обратиться к API. Например:

- `GET /api/notes` — получить список всех своих заметок в JSON (только заголовки и ID)
- `GET /api/note/42` — получить полный текст заметки с вложениями
- `POST /api/note` — создать заметку, передав JSON `{"title": "...", "content": "..."}`
- `PUT /api/note/42` — обновить
- `DELETE /api/note/42` — удалить

Всё это работает только после того, как пользователь залогинен (используется cookie сессии, как в браузере). Это значит, что через тот же API можно управлять заметками из

консоли браузера, из скрипта на Python или из Telegram-бота.

Что можно добавить в будущем (перспективы)

- Поиск по заметкам (прямо на главной)
- Счётчик заметок пользователя
- Возможность сделать заметку «публичной» (доступной без авторизации по прямой ссылке)
- Экспорт сразу всех заметок ZIP-архивом

Реализация

1. Сущности БД (SQLAlchemy ORM)

User

Поле	Тип	Описание
id	Integer PK	ID пользователя
username	String(64), unique	Логин
password_hash	String(128)	Хеш пароля (Flask-Bcrypt или werkzeug)
created_at	DateTime	Дата регистрации

Note

Поле	Тип	Описание
id	Integer PK	ID заметки
title	String(200)	Заголовок
content	Text	Текст заметки (в Markdown)
created_at	DateTime	Дата создания
updated_at	DateTime	Дата обновления
user_id	Integer FK(User.id)	Владелец

Attachment (опционально, но для «загрузки файлов» нужно)

Поле	Тип	Описание
id	Integer PK	ID файла
filename	String(200)	Оригинальное имя
stored_name	String(200)	Уникальное имя на диске
file_type	String(50)	image/png, image/jpeg и т.д.
note_id	Integer FK(Note.id)	К какой заметке прикреплён

Связи: User 1→N Note, Note 1→N Attachment

2. Маршруты (Flask)

Публичные

- GET /register — форма регистрации
- GET /login — форма входа
- GET /logout — выход

Приватные (только авторизованные)

- GET / → редирект на /notes
- GET /notes — список своих заметок (сортировка по updated_at)
- GET /note/<id> — просмотр одной заметки (с вложениями)
- GET /note/create — форма создания
- POST /note/create — создать заметку
- GET /note/<id>/edit — форма редактирования
- POST /note/<id>/edit — обновить
- POST /note/<id>/delete — удалить заметку

Для вложений

- POST /note/<id>/attach — загрузить файл
- GET /attachment/<id> — скачать/показать файл
- POST /attachment/<id>/delete — удалить вложение

Экспорт (ключевая фишка)

- GET /note/<id>/export/md — скачать как .md (Markdown)

- GET `/note/<id>/export/html` — скачать как `.html`
- GET `/note/<id>/export/txt` — скачать как `.txt` (простой текст)

В HTML-экспорте можно на лету конвертировать Markdown → HTML (через `markdown` библиотеку), подложить простой CSS и сохранить как полноценный HTML-файл.

3. REST API (для тренировки)

Эндпоинты (JSON):

- GET `/api/notes` — вернуть список всех своих заметок (только `id`, `title`, `updated_at`)
- GET `/api/note/<id>` — вернуть одну заметку включая `content`
- POST `/api/note` — создать заметку (JSON: `title`, `content`)
- PUT `/api/note/<id>` — обновить
- DELETE `/api/note/<id>` — удалить

Все API-методы требуют отправки токена (можно начать с сессии: `session["user_id"]`), а в перспективе — `flask-jwt-extended`).

4. Bootstrap интерфейс (минимальный, но удобный)

Используем **Bootstrap 5** (CDN). Основные страницы:

- **Вход / регистрация** — карточка по центру
- **Список заметок** — таблица или сетка карточек, кнопки: Edit, Delete, Export (dropdown с `md/html/txt`)
- **Форма заметки** — `textarea` для поддержки Markdown (можно без live-превью на старте, просто поле)
- **Просмотр заметки** — Markdown преобразуется в HTML (библиотека `markdown`). Внизу — список файлов.

5. Примерная структура проекта

```
NoteTrackerExport/  
├─ app.py           # главный запуск  
├─ models.py        # User, Note, Attachment  
├─ forms.py         # WTForms (если используешь)
```

```
├─ routes/                # можно разбить
│   ├─ auth.py
│   ├─ notes.py
│   ├─ api.py
│   └─ export.py
├─ templates/
│   ├─ base.html          # с navbar Bootstrap
│   ├─ login.html
│   ├─ register.html
│   ├─ notes_list.html
│   ├─ note_form.html
│   └─ note_view.html
├─ static/                # (почти не нужен, Bootstrap на CDN)
├─ uploads/              # загруженные файлы (игнорируем в git)
├─ instance/
│   └─ app.db             # SQLite база
└─ requirements.txt
```